# Lesson 6 Teacher's Key

Godot Engine

**INSTRUCTOR ONLY - do not distribute to students**

## Level 1 - Complete Worked Solution

A custom signal on enemy.tscn, connected once per instance at spawn time to a dedicated GameManager node's script, which owns a hidden Label shown at zero:

```
# enemy.tscn's script:
signal destroyed

func _on_hitbox_area_entered(area):
    area.get_parent().queue_free()
    destroyed.emit()
    queue_free()

# GameManager's script (GameManager is a child Node of the main scene,
sibling of WinText):
var enemies_remaining := 5

func on_enemy_destroyed():
    enemies_remaining -= 1
    if enemies_remaining <= 0:
        get_node("../WinText").show()

# Main scene's script:
var enemy_scene := preload("res://enemy.tscn")

func _ready():
    for enemy_count in range(5):
        var new_enemy := enemy_scene.instantiate()
        new_enemy.position = Vector2(40 + enemy_count * 60, 40)
        new_enemy.destroyed.connect($GameManager.on_enemy_destroyed)
        add_child(new_enemy)
```

## Level 2 - Complete Worked Solution

destroyed.emit() fires exactly once per enemy, right where it's actually destroyed, and each enemy's signal is connected exactly once at spawn time, always to the same GameManager method - there's no way for enemies_remaining to drift or for WinText to show more than once.

## Level 3 - All Worked Solutions

Every stretch-goal idea from the handout, fully worked - not just one. Useful for validating each idea actually works before class, and as a ready reference no matter which one a student ends up picking.

### Sound: play on win

Add a plain AudioStreamPlayer (non-2D, since it's UI) as a child of GameManager, with the provided sfx_win.wav assigned as its stream:

```
# GameManager's script:
func on_enemy_destroyed():
    enemies_remaining -= 1
    if enemies_remaining <= 0:
        get_node("../WinText").show()
        $WinSound.play()
```

### Visual: animate the win text in

A quick scale tween reads more like a celebration than a plain show():

```
# GameManager's script:
func on_enemy_destroyed():
    enemies_remaining -= 1
    if enemies_remaining <= 0:
        var win_text := get_node("../WinText")
        win_text.show()
        win_text.scale = Vector2(0.5, 0.5)
        var tween := create_tween()
        tween.tween_property(win_text, "scale", Vector2(1, 1), 0.3)
```

### Structure: GameManager owns the counting

enemy.tscn only ever announces that it was destroyed (destroyed.emit()), and the main scene only ever spawns enemies and wires up their signals - all counting and the win-state decision live entirely inside GameManager's own script.

### Complete Level 3 solution (all 3 goals combined)

Everything above combined, in GameManager's script:

```
func on_enemy_destroyed():
    enemies_remaining -= 1
    if enemies_remaining <= 0:
        var win_text := get_node("../WinText")
        win_text.show()
        win_text.scale = Vector2(0.5, 0.5)
        var tween := create_tween()
        tween.tween_property(win_text, "scale", Vector2(1, 1), 0.3)
        $WinSound.play()
```